

Source Code of App: Dashborad

admin.py

```
from django.contrib import admin

# Register your models here.
```

apps.py

```
from django.apps import AppConfig

class DashboradConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "Dashborad"
```

tests.py

```
from django.test import TestCase

# Create your tests here.
```

urls.py

```
from django.contrib import admin

from django.urls import path , include
from .views import *
app_name = 'Dashboard'
urlpatterns = [
    path('Dashboard/', DashboardHomeView.as_view(), name='dashboard_home'),
    path('dashboard/data/', DashboradDataView.as_view(), name='dashboard-data'),
    path('dashboard/weekly-usage-stats/', WeeklyUsageStatsView.as_view(), name='weekly_status'),
    path('dashboard/yesterday-usage-stats/', YesterdayDataView.as_view(), name='Yesterday_status'),
    path('dashboard/weekly-average-scores/', WeeklyAverageScoresView.as_view(), name='Yesterday_status'),
    path('dashboard/today-usage-history/', TodayUsageHistoryView.as_view(), name='Yesterday_status'),
    path('dashboard/heatmap-data/', HeatmapDataView.as_view(), name='heatmap_data'),
    path('detailed-usage-data/', DetailedUsageDataView.as_view(), name='detailed-usage-data'),
]
```

views.py

```
from django.shortcuts import render
from django.http import JsonResponse
from django.utils.timezone import make_aware
from django.db.models.functions import ExtractWeekDay
from django.db.models import Avg
from django.contrib.auth.mixins import LoginRequiredMixin
from base.models import *
from django.views.generic import TemplateView, View
from django.db.models.functions import TruncHour, TruncMinute
from django.db.models import Count , F
from collections import defaultdict
from datetime import date , timedelta , datetime
from django.utils.timezone import now, localtime
# Create your views here.

GOAL_DESCRIPTIONS = {
    'Improve posture': 'พัฒนามคัการนั่งให้เหมาะสม และ ดีต่อสุขภาพ',
    'Health maintenance': 'เพื่อสุขภาพที่ยั่งยืน อากการปวดหลังลดน้อยลง',
    'Posture correction': 'เพื่อนั่งให้ได้อย่างถูกต้อง',
}

class DashboardHomeView(TemplateView):
    template_name = 'Dashboard.html'
    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        user = self.request.user

        user_info = UserInfomation.objects.filter(username=user.username).first()
        usage_history = UserUsageHistory.objects.filter(posture_detection__user=user_info)
        posture_data = PostureDetection.objects.filter(user=user_info)
        personal_info = PersonalInformation.objects.filter(user=user_info).first()

        usage_count = posture_data.count()

        Rank = user_info.role if user_info else "ไม่มีข้อมูล Role"

        context['usage_history'] = usage_history
        context['posture_data'] = posture_data

        # Calculate daily scores
        daily_scores = defaultdict(list)
        for entry in posture_data:
            day = entry.timestamp.date()
            daily_scores[day].append(entry.score)
```

```

today = date.today()

start_of_day = datetime.combine(today, datetime.min.time())
end_of_day = datetime.combine(today, datetime.max.time())

scores_today = PostureDetection.objects.filter(timestamp__range=(start_of_day, end_of_day))

# คำนวณช่วงเวลาที่ใช้งานน้อยที่สุดสำหรับวันนี้
peak_time = (
    scores_today.annotate(hour=TruncHour('timestamp')) # ตัดช่วงเวลาเป็นรายชั่วโมง
    .values('hour') # รวมกลุ่มตามชั่วโมง
    .annotate(count=Count('id')) # นับจำนวน
    .order_by('-count') # เรียงลำดับจากมากไปน้อย
)

if peak_time.exists():
    peak_hour = peak_time[0]['hour']
    peak_hour_count = peak_time[0]['count']
else:
    peak_hour = None
    peak_hour_count = 0

# Calculate today's weighted average score
total_score = 0
total_duration = 0
for entry in scores_today:
    if entry.userusagehistory_set.exists():
        duration = entry.userusagehistory_set.first().detection_time
        if duration is None:
            duration = 0
    else:
        duration = 0
    total_score += entry.score * duration
    total_duration += duration

avg_score_today = total_score / total_duration if total_duration > 0 else 0

goal = personal_info.goal if personal_info else "ผู้ใช้งานยังไม่ได้เลือกเป้าหมาย"

tooltip_text = GOAL_DESCRIPTIONS.get(goal, "ผู้ใช้งานยังไม่ได้เลือกเป้าหมาย")

context['goal'] = goal
context['tooltip_text'] = tooltip_text

yesterday = date.today() - timedelta(days=1)

# Filter records for yesterday
yesterday_usage = PostureDetection.objects.filter(
    timestamp__date=yesterday,
    user=user
)

# Group scores by day
daily_scores = defaultdict(list)
for entry in yesterday_usage:
    day = entry.timestamp.date()
    daily_scores[day].append(entry.score)

# Calculate yesterday's weighted average score
total_score_yesterday = 0
total_duration_yesterday = 0
for entry in yesterday_usage:
    if entry.userusagehistory_set.exists():
        duration = entry.userusagehistory_set.first().detection_time if entry.userusagehistory_set.exists() else 0
        if duration is None:
            duration = 0
    else:
        duration = 0

    total_score_yesterday += entry.score * duration
    total_duration_yesterday += duration

avg_score_yesterday = total_score_yesterday / total_duration_yesterday if total_duration_yesterday > 0 else 0

if total_duration_yesterday > 0:
    max_yesterday = max([entry.score for entry in yesterday_usage])
    min_yesterday = min([entry.score for entry in yesterday_usage])
else:
    max_yesterday = 0
    min_yesterday = 0

if total_duration > 0:
    max_today = max([entry.score for entry in scores_today])
    min_today = min([entry.score for entry in scores_today])
else:
    max_today = 0
    min_today = 0

context['usage_count'] = usage_count
context['yesterday_score'] = avg_score_yesterday
context['max_yesterday'] = max_yesterday
context['min_yesterday'] = min_yesterday

```

```

context['Rank'] = Rank
context['avg_score'] = avg_score_today
context['today_count'] = len(scores_today)
context['peak_hour'] = peak_hour
context['peak_count'] = peak_hour_count
context['max_score'] = max_today
context['min_score'] = min_today

return context

class DashboradDataView(View):
    def get(self, request, *args, **kwargs):
        user = request.user

        posture_data = PostureDetection.objects.filter(user=user)

        # Calculate daily scores
        daily_scores = defaultdict(list)
        for entry in posture_data:
            day = entry.timestamp.date()
            daily_scores[day].append(entry.score)

        # Calculate today's average score
        today = date.today()
        scores_today = daily_scores.get(today, [])
        avg_score_today = sum(scores_today) / len(scores_today) if scores_today else 0

        # Prepare JSON response
        data = {
            'categories': ['Score', 'Remaining'],
            'values': [avg_score_today, 100 - avg_score_today],
        }
        return JsonResponse(data)

class YesterdayDataView(View):
    def get(self, request, *args, **kwargs):
        user = request.user

        # Calculate yesterday's date
        yesterday = date.today() - timedelta(days=1)

        # Filter records for yesterday
        yesterday_usage = PostureDetection.objects.filter(
            timestamp__date=yesterday,
            user=user
        )

        # Group scores by day
        daily_scores = defaultdict(list)
        for entry in yesterday_usage:
            day = entry.timestamp.date()
            daily_scores[day].append(entry.score)

        # Calculate yesterday's average score
        scores_yesterday = daily_scores.get(yesterday, [])
        avg_score_yesterday = (
            sum(scores_yesterday) / len(scores_yesterday)
            if scores_yesterday
            else 0
        )

        data = {
            'categories': ['Score', 'Remaining'],
            'values': [avg_score_yesterday, 100 - avg_score_yesterday],
        }

        return JsonResponse(data)

class WeeklyUsageStatsView(View):
    def get(self, request, *args, **kwargs):
        user = request.user

        start_of_week = date.today() - timedelta(days=date.today().weekday())
        end_of_week = start_of_week + timedelta(days=6)

        weekly_usage = UserUsageHistory.objects.filter(
            posture_detection__timestamp__date_gte=start_of_week,
            posture_detection__timestamp__date_lte=end_of_week,
            posture_detection__user=user
        ).annotate(weekday=ExtractWeekDay('posture_detection__timestamp')).values('weekday').annotate(count=Count('id'))

        weekday_map = {
            2: 'จันทร์', 3: 'อังคาร', 4: 'พุธ', 5: 'พฤหัสบดี', 6: 'ศุกร์', 7: 'เสาร์', 1: 'อาทิตย์'
        }
        usage_data = {weekday_map[i]: 0 for i in range(1, 8)} # เริ่มจากค่าเริ่มต้น 0

        for item in weekly_usage:
            usage_data[weekday_map[item['weekday']]] = item['count']

        return JsonResponse({'usage_data': usage_data})

```

```

class HeatmapDataView(LoginRequiredMixin, View):
    def get(self, request, *args, **kwargs):
        # 1. Query ข้อมูลการใช้งาน แยกตามวันและชั่วโมง
        usage_data = PostureDetection.objects.annotate(
            day_of_week=F('timestamp__week_day'), # วันในสัปดาห์ (1=Sunday, 7=Saturday)
            hour_of_day=F('timestamp__hour') # ชั่วโมง
        ).values('day_of_week', 'hour_of_day').annotate(count=Count('id'))

        # 2. Mapping ข้อมูลให้อยู่ในรูปแบบ Heatmap
        days_mapping = {
            1: 'Sunday', 2: 'Monday', 3: 'Tuesday', 4: 'Wednesday',
            5: 'Thursday', 6: 'Friday', 7: 'Saturday'
        }
        time_ranges = {
            'กลางคืน': range(0, 6),
            'เช้า': range(6, 12), # 6:00 - 11:59
            'สาย': range(12, 16), # 12:00 - 15:59
            'บ่าย': range(16, 20), # 16:00 - 19:59
            'เย็น': range(20, 24) # 20:00 - 23:59
        }

        series = []
        for day in range(1, 8): # Loop ทุกวันในสัปดาห์
            day_data = {'name': days_mapping[day], 'data': []}
            for time_range_name, hours in time_ranges.items(): # Loop ทุกช่วงเวลา
                count = sum(
                    entry['count']
                    for entry in usage_data
                    if entry['day_of_week'] == day and entry['hour_of_day'] in hours
                )
                day_data['data'].append({'time_range': time_range_name, 'count': count})
            series.append(day_data)

        return JsonResponse({'series': series, 'days': list(days_mapping.values())})

class WeeklyAverageScoresView(View):
    def get(self, request, *args, **kwargs):
        user = request.user

        # คำนวณช่วงเวลาในสัปดาห์นี้
        start_of_week = date.today() - timedelta(days=date.today().weekday())
        end_of_week = start_of_week + timedelta(days=6)

        # ดึงข้อมูลการใช้งานในสัปดาห์นี้
        weekly_scores = PostureDetection.objects.filter(
            timestamp__date__gte=start_of_week,
            timestamp__date__lte=end_of_week,
            user=user
        ).annotate(weekday=ExtractWeekDay('timestamp')).values('weekday').annotate(avg_score=Avg('score'))

        weekday_map = {1: 'SUN', 2: 'MON', 3: 'TUE', 4: 'WED', 5: 'THU', 6: 'FRI', 7: 'SAT'}
        usage_data = {weekday_map[i]: 0 for i in range(1, 8)} # เริ่มต้นข้อมูลทุกวันด้วย 0
        for item in weekly_scores:
            usage_data[weekday_map[item['weekday']]] = round(item['avg_score'], 2) # เก็บค่าเฉลี่ยโดยปัดทศนิยม 2 ตำแหน่ง

        return JsonResponse({'usage_data': usage_data})

class TodayUsageHistoryView(View):
    def get(self, request, *args, **kwargs):
        user = request.user

        # ดึงข้อมูลของวันนี้
        today = date.today()
        today_data = PostureDetection.objects.filter(
            timestamp__date=today,
            user=user
        ).select_related('userusagehistory').values(
            'timestamp',
            'score',
            'userusagehistory__detection_time'
        )

        # แปลงข้อมูลให้อยู่ในรูปแบบ JSON
        history_data = [
            {
                "time": item['timestamp'].strftime('%H:%M'),
                "duration": str(item['userusagehistory__detection_time']) if item['userusagehistory__detection_time'] else None,
                "score": item['score']
            }
            for item in today_data
        ]

        return JsonResponse({'history': history_data})

class DetailedUsageDataView(View):
    def get(self, request, *args, **kwargs):
        user = request.user
        today = date.today()

```

```

# ดึงข้อมูลการใช้งานของวันนี้
usage_data = UserUsageHistory.objects.filter(
    posture_detection__timestamp__date=today,
    posture_detection__user=user
).select_related('posture_detection').order_by('posture_detection__timestamp')

# เตรียมข้อมูล
results = []
for entry in usage_data:
    posture = entry.posture_detection # ดึง PostureDetection ที่เกี่ยวข้อง
    start_time = posture.timestamp
    duration = entry.detection_time if entry.detection_time else 0
    end_time = start_time + timedelta(seconds=duration)
    score = posture.score # ดึงคะแนนจาก PostureDetection

    results.append({
        'start_time': start_time.strftime('%H:%M:%S'),
        'end_time': end_time.strftime('%H:%M:%S'),
        'score': score
    })

return JsonResponse({'usage_data': results})

```

__init__.py

__init__.py

Dashboard.html

```

{% extends 'base.html' %}

{% block main %}

```

Dashboard

สรุปการใช้งานในวันนี้ในรูปแบบของรายชั่วโมง

ความคืบหน้า

เป้าหมาย
 {{ goal }}

จำนวนครั้งที่ใช้งาน
 {{ usage_count }} ครั้ง

Ranking
 {{ Rank }}

การใช้งานในวันนี้

คะแนนมากที่สุด
{{ max_score }}

คะแนนน้อยที่สุด
{{ min_score }}

กราฟสรุปคะแนนเฉลี่ยในสัปดาห์

ประวัติการใช้งานเมื่อวาน

คะแนนมากที่สุด
{{ max_yesterday }}

คะแนนน้อยที่สุด
{{ min_yesterday }}

สถิติการใช้งานในสัปดาห์นี้

ความถี่ของการใช้งานในแต่ละวัน

{% endblock %}